

República Bolivariana de Venezuela
Ministerio del Poder Popular para la Educación Universitaria
Universidad Politécnica Territorial
´´José Antonio Anzoátegui´´
Trayecto 2 Fase 2 PNF: Informática
Municipio Guanta, Edo. Anzoátegui

PROGRAMACIÓN II

Estudiantes:
Wuindel González
C.I: 26.385.803

Ing. Humner Lenis.

Guanta, 7 de Noviembre de 2026

Definición y Fundamentos del Software.

El término software hace referencia al conjunto de componentes lógicos intangibles que hacen posible la realización de tareas específicas en un sistema informático. En contraposición al hardware (la parte física formada por circuitos, procesadores y memorias), el software abarca la totalidad de las instrucciones, programas, reglas, documentación y datos asociados que operan sobre el sistema informático.

Desde una perspectiva formal según la IEEE (Institute of Electrical and Electronics Engineers), el software se define como el conjunto de los programas de cómputo, procedimientos, reglas y documentación asociada, relativas a la operación de un sistema de computación.

Clasificación del Software

- **Software de Sistema:** Es el conjunto de programas que administran y gestionan los recursos del hardware, sirviendo de interfaz entre los componentes físicos y los programas de aplicación. Incluye los sistemas operativos (Windows, Linux, macOS), los controladores de dispositivos (drivers), firmware y herramientas de diagnóstico.
- **Software de Aplicación:** Comprende todos aquellos programas diseñados para ayudar al usuario a realizar tareas específicas de carácter personal, profesional o recreativo. Ejemplos representativos son procesadores de texto, hojas de cálculo, navegadores web, sistemas de gestión empresarial (ERP) y editores de diseño gráfico.
- **Software de Programación (o Desarrollo):** Corresponde al conjunto de herramientas utilitarias que permiten al desarrollador o programador escribir, probar, depurar y mantener otros programas informáticos. Incluye compiladores, intérpretes, entornos de desarrollo integrados (IDE), depuradores (debuggers) y editores de código.

Definición y Paradigmas de la Programación

La programación es el arte y la ciencia de codificar un algoritmo en un lenguaje inteligible para una computadora, con el fin de resolver un problema determinado o ejecutar una tarea específica. Involucra un proceso creativo y estructurado que abarca el diseño, la escritura, la verificación, el formateo, la depuración y el mantenimiento del código fuente.

Lenguajes de Programación y Niveles de Abstracción

Los lenguajes de programación son sistemas formales compuestos por un conjunto de reglas sintácticas y semánticas que permiten instruir a un computador. Se clasifican según su nivel de abstracción respecto al hardware:

- **Lenguajes de Bajo Nivel:** Están íntimamente ligados a la arquitectura física del procesador. Destaca el Código Máquina (instrucciones binarias 0 y 1) y el Lenguaje Ensamblador (Assembly), que utiliza mnemónicos directos de CPU.

- **Lenguajes de Alto Nivel:** Diseñados para ser comprensibles por los seres humanos, haciendo uso de palabras estructuradas (habitualmente en inglés) y notación matemática. Ejemplos: C, C++, Java, Python, PHP, JavaScript.

Programación Estructurada

La Programación Estructurada surgió a finales de la década de 1960 como una respuesta a la crisis del software, caracterizada por códigos inmanejables, difíciles de leer y propensos a errores masivos debidos al uso indiscriminado de saltos incondicionales (la instrucción GOTO), lo que generaba el denominado 'código espagueti'.

Pioneros como Edsger Dijkstra argumentaron que la calidad y claridad de un programa se incrementan radicalmente al restringir el flujo de ejecución a estructuras lógicas bien definidas con un único punto de entrada y un único punto de salida.

Teorema de la Programación Estructurada

Formulado por Corrado Böhm y Giuseppe Jacopini en 1966, este teorema constituye el pilar teórico del paradigma. Establece que cualquier programa ejecutable o función computable puede ser redactado utilizando únicamente tres estructuras de control básicas:

- **Secuencia:** Ejecución sucesiva de instrucciones en el orden estricto en que aparecen escritas.
- **Selección (o Decisión):** Bifurcación del flujo según la evaluación de una condición lógica (estructuras Si-Entonces / Si-Entonces-Sino).
- **Iteración (o Repetición):** Repetición de un bloque de código mientras se cumpla o hasta que se satisfaga una determinada condición lógica.

Ventajas de la Programación Estructurada

Entre sus principales beneficios se destacan: la reducción drástica de errores, la facilidad para leer y mantener el código, la posiblidad de dividir problemas complejos en módulos más simples (diseño descendente o Top-Down) y la simplicidad en la realización de pruebas unitarias y depuración.

Algoritmos: La Base Lógica

Definición y Propiedades de un Algoritmo

Un algoritmo es una secuencia ordenada, finita y unívoca de pasos o instrucciones bien definidas que permiten resolver un problema, realizar un cálculo o procesar datos. Constituye el esquema abstracto previo a la codificación en cualquier lenguaje de programación.

Todo algoritmo riguroso debe cumplir con las siguientes características fundamentales:

- **Finitud:** Un algoritmo debe terminar siempre tras un número finito de pasos.
- **Definición (Precisión):** Cada paso debe estar especificado de forma clara y sin ambigüedades.
- **Entrada:** Posee cero o más elementos de entrada suministrados antes de la ejecución.
- **Salida:** Produce uno o más resultados vinculados con la entrada inicial.
- **Efectividad:** Las operaciones deben ser lo suficientemente básicas para ser ejecutadas en tiempo finito.

Métodos de Representación

Los algoritmos pueden expresarse mediante diversos lenguajes de representación priorizando la claridad conceptual:

- **Pseudocódigo:** Lenguaje informal de alto nivel que combina léxico humano con estructuras sintácticas de programación.
- **Diagramas de Flujo:** Representación gráfica esquemática mediante símbolos estandarizados (rectángulos, rombos, óvalos) unidos por flechas de flujo.

Estructuras de Datos: Los Arrays (Vectores y Matrices)

Definición de Array o Arreglo

Un array (conocido también como arreglo, vector o matriz) es una estructura de datos homogénea y contigua en memoria que permite almacenar una colección finita de elementos del mismo tipo bajo un único nombre de variable. Los elementos son identificados de forma individual mediante uno o más índices numéricos enteros.

Tipos de Arrays

Según la cantidad de dimensiones utilizadas para, acceder a sus celdas, los arrays se clasifican en:

- **Unidimensionales (Vectores):** Estructuras de una sola dimensión. Se accede a cada elemento mediante un solo índice, por ejemplo: arreglo[0]. Son ideales para representar listas secuenciales de datos (nombres, temperaturas diarios).
- **Multidimensionales (Matrices y Tablas):** Estructuras organizadas en filas y columnas (bidimensionales) o en tres o más ejes. Para acceder a una posición en una matriz bidimensional se requieren dos índices: matriz[fila][columna].

Operaciones Fundamentales sobre Arrays

Las operaciones clásicas sobre arreglos incluyen el recorrido (traversal) para inspeccionar sus valores, la búsqueda de elementos (búsqueda lineal o binaria), la ordenación (mediante algoritmos como Burbuja, Quicksort o Merge Sort) y la actualización de posiciones específicas.

Estructuras de Control Repetitivas: Los Bucles

Concepto de Bucle o Iteración

Un bucle (o ciclo/iteración) es una estructura de control que permite ejecutar de forma reiterada un determinado bloque de instrucciones mientras una condición lógica dada se mantenga verdadera. Los bucles son indispensables para automatizar tareas repetitivas y procesar colecciones masivas de datos almacenadas en estructuras como los arrays.

Clasificación de Bucles

Existen tres variedades fundamentales de estructuras repetitivas en la programación estructurada:

- **Bucle Mientras (While):** Evalúa la condición booleana AL INICIO del ciclo. Si la condición es falsa desde un principio, el cuerpo del bucle no se ejecuta ni una sola vez.
- **Bucle Hacer-Mientras (Do-While):** Evalúa la condición AL FINAL del ciclo. Esto garantiza que las instrucciones contenidas en el bloque se ejecuten al menos una vez, independientemente de la condición.
- **Bucle Para (For):** Diseñado para iteraciones donde el número de repeticiones se conoce de antemano. Integra en una sola línea la inicialización del contador, la condición de parada y el incremento/decremento.

Errores Comunes: Bucles Infinitos

Un fallo crítico en la programación con bucles ocurre cuando la condición de parada nunca se evalúa a falsa, dando lugar a un 'bucle infinito'. Este evento agota los recursos de memoria y procesador de la máquina, bloqueando la ejecución del programa.

Modularidad y Reutilización: Las Funciones

Concepto de Función y Subrutina

Una función es un bloque de código autónomo, independiente y reutilizable diseñado para realizar una tarea específica dentro de un programa. Las funciones reciben datos de entrada (parámetros), procesan dicha información y devuelven un resultado (valor de retorno) al punto donde fueron convocadas o invocadas.

Parámetros, Argumentos y Retorno de Valores

La firma de una función especifica los datos requeridos. Los 'parámetros' representan las variables declaradas en la definición de la función, mientras que los 'argumentos' son los valores reales transmitidos a la función durante la llamada.

Paso por Valor vs. Paso por Referencia

La transferencia de datos a una función se realiza mediante dos mecanismos clave:

- **Paso por Valor:** Se pasa una copia exacta del dato a la función. Las modificaciones realizadas a la variable dentro de la función no afectan a la variable original fuera de ella.
- **Paso por Referencia:** Se pasa la dirección de memoria de la variable. Cualquier cambio efectuado en el parámetro dentro de la función alterará de forma directa y permanente la variable original.